ECE 441 MAJOR DESIGN LAB EXPERIMENT #4

Human-Machine Interface: Cloud Database and Mobile Device

1. Objectives

In this experiment, the students will learn how to integrate a cloud database to their embedded Systems, and accessing the database remotely on an Android cellphone, thus achieving the 'connected' feature of a Smart & Connected Embedded System.

As shown in Figure 1, A simple embedded system is used in this experiment for demonstration. In this system, an AHT-20 humidity and temperature sensor provides sensor data to the Raspberry Pi; the Raspberry Pi then uploads the data to the cloud database through the Internet; then a cellphone can access the sensor data from the cloud database remotely.

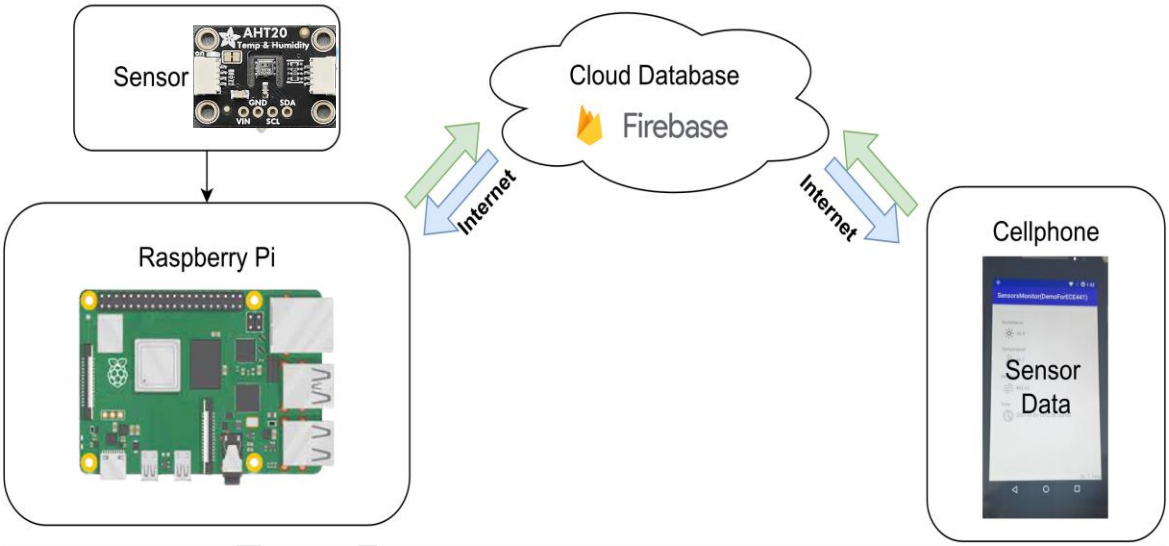


Figure 1. A connected Embedded System

2. Components

- 1 x Raspberry Pi
- 1 x AHT-20 Temperature and Humidity Sensor
- 1 x Breadboard
- 1x Android Phone(Optional)

3. Background

A. Cloud database

A cloud database is a database that typically runs on a cloud computing platform and access to the database is provided as-a-service. There are two common deployment models: users can

run databases on the cloud independently, using a virtual machine image, or they can purchase access to a database service, maintained by a cloud database provider. Of the databases available on the cloud, some are SQL-based and some use a NoSQL data model. Next, we will talk about a specific cloud database which we will use in this lab.

B. Firebase Realtime Database

The Firebase Realtime Database is a cloud-hosted database. Data is stored as JSON and synchronized in realtime to every connected client. When building cross-platform apps with Apple platforms, Android, and JavaScript SDKs, all of the clients share one Realtime Database instance and automatically receive updates with the newest data. Those features make it especially suitable for an embedded system.

The Firebase Realtime Database also has some other nice features such as built-in cybersecurity for an extra layer of security, a free cloud messaging service, and the support for google analytics for app marketing and performance optimizations.

C. Android

Android is a mobile operating system based on a modified version of the Linux kernel and other open-source software, designed primarily for touchscreen mobile devices such as smartphones and tablets. Android is developed by a consortium of developers known as the Open Handset Alliance and commercially sponsored by Google. Although manufacturers modify the Android system for their own devices, the compatibility is usually maintained.

D. Android SDK

Android SDK is the software tool kit used for development of Android applications. It provides a library of APIs (application programming interface) to interact with the hardware of the target Android devices. During development, the Android emulator can be utilized for emulation on the target platforms, so that the developers can test their applications on a wide range of target devices without owning them.

Android SDK natively support Java, although an extension toolkit called Android NDK can be used for C/C++ support during the development.

4. Preliminary Works

Prepare for the lab by following the instructions in this section.

1. Check the documentation for Firebase, and create an account.
2. Install Android Studio, or any other alternatives if preferred.

5. Experiment Procedures

A. Create a Firebase Real-time Database

1. Navigate to the Realtime Database section of the Firebase console. You'll be prompted to select an existing Firebase project. Follow the database creation workflow.
2. Select the Test mode for the database
3. Choose a location for the database. Whichever is closest to you.
4. Click Done.

Not for Distribution

B. Set up the Raspberry Pi for Sensor Data Collection

1. Refer to the wiring diagram provided in figure 2 to connect the AHT-20 sensor to the Raspberry Pi.

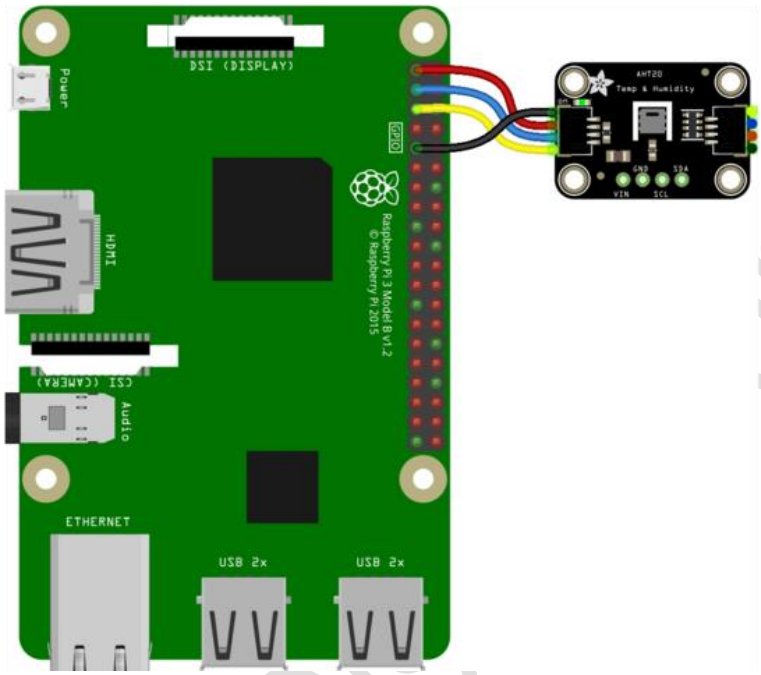


Figure 2. Wiring diagram

2. Install required libraries with the commands :

```
pip3 install pyrebase4 -U --break-system-packages
pip3 install adafruit-blinka -U --break-system-packages
pip3 install adafruit-circuitpython-ahtx0 -U --break-system-packages
sudo apt install libgpiod-dev
```

3. Run the script provided in **Appendix A**, you should see the console output on the Raspberry as shown in figure 3.

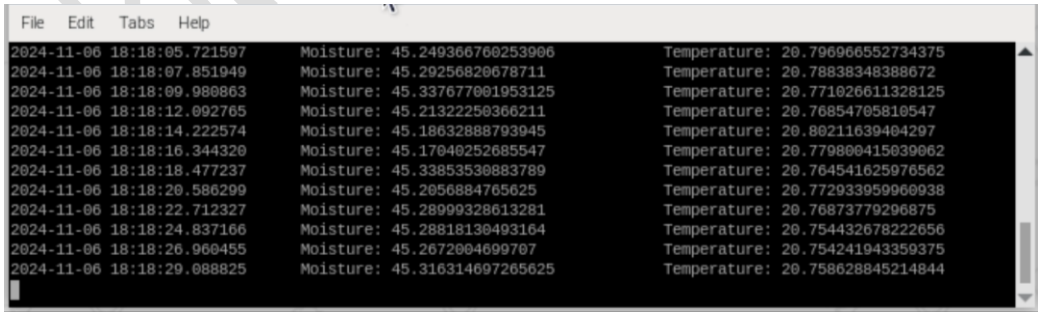
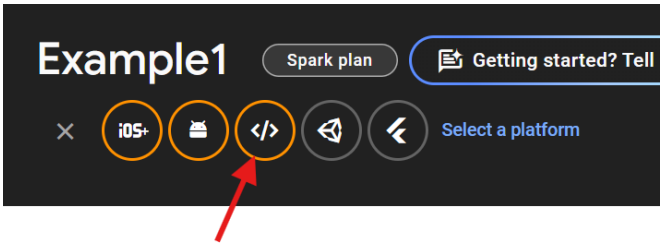
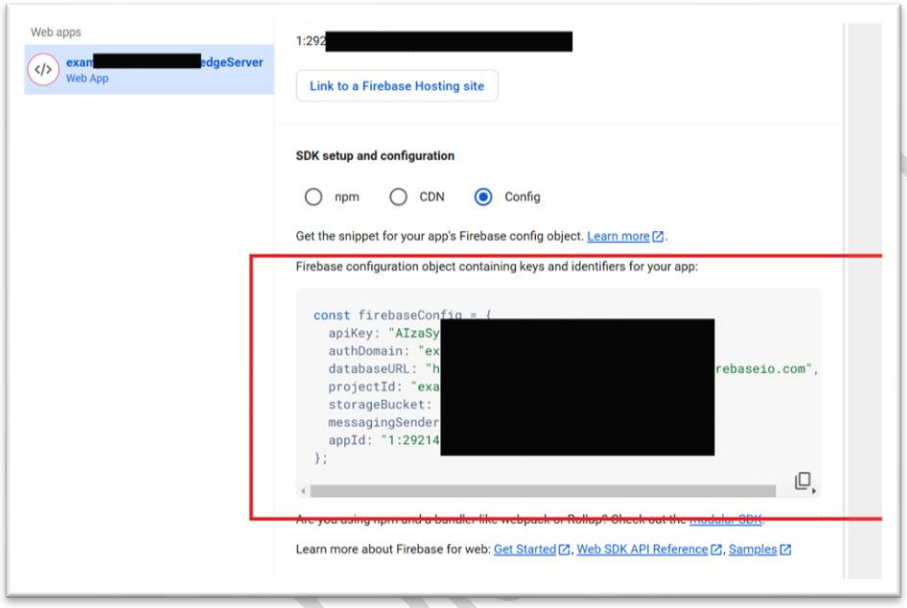


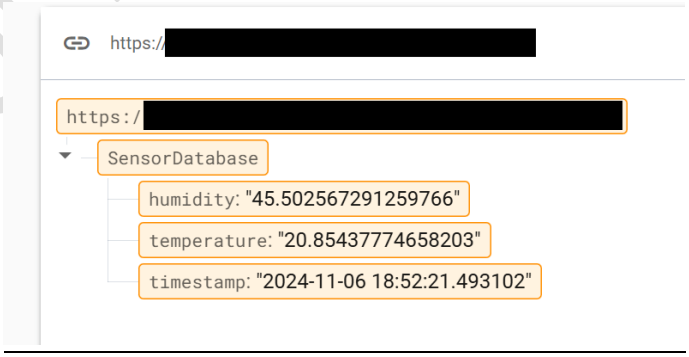
Figure 3. Sensor data output on the Raspberry Pi

4. Then add a web app in the firebase console, and copy the data from the config block to the python script Raspberry Pi.



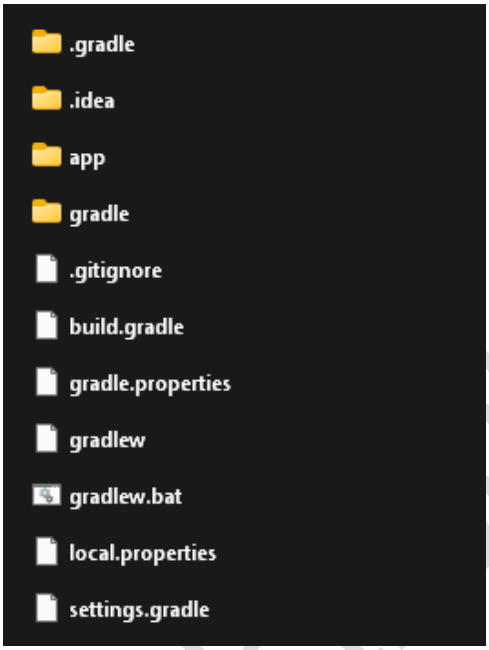


5. Uncommont the line containing `database.child`, which is responsible for updating the database. The script will upload the sensor data to the cloud, so make sure the Raspberry Pi has internet access. Open your Realtime database from the Firebase Dashboard. If all things go well, the sensor data will appear in the Realtime database.

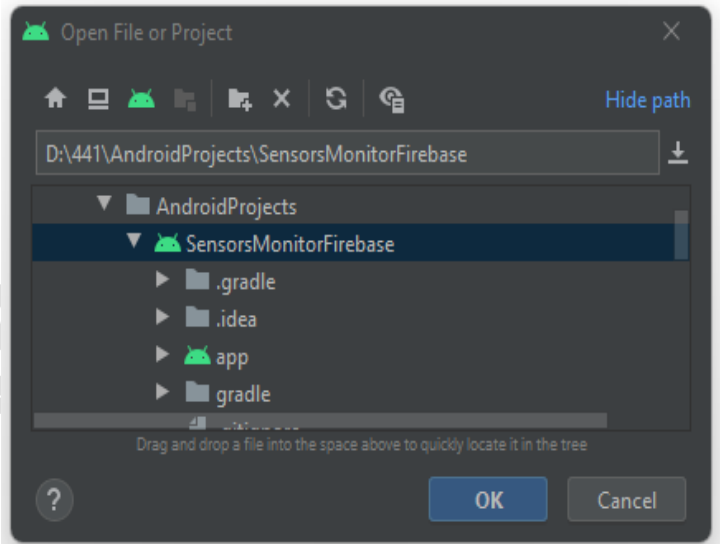


C. Synchronizing the sensor data on the Android application

The application is packaged in the zip file attached with this lab documentation. Extract the zip file into a new folder, you should see the following files:



Now open Android Studio, go to File/Open.., and select the folder we just extracted into:



This will open the project browser. Check out the main source code under 'app\src\main\java\com.example.sensormonitorfirebase' and try to understand the script and comment on it. This script synchronizes the data from the cloud database and display it in the UI.

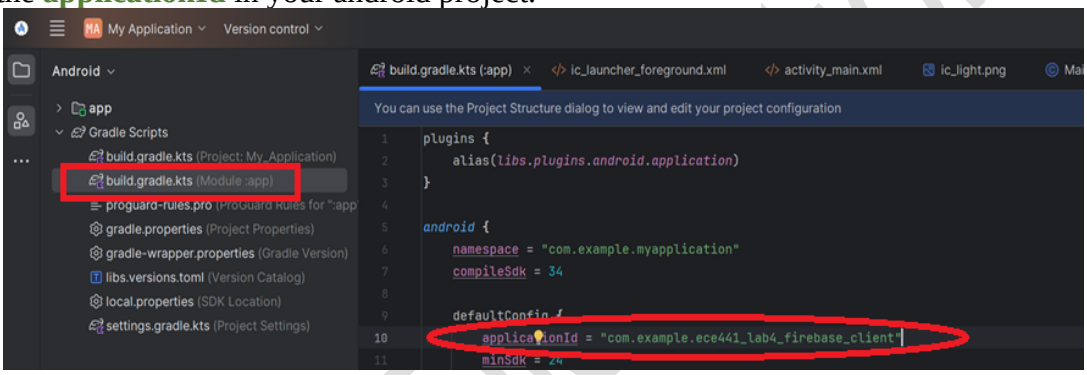
This project is completed out of the box, but you will need to connect it to your own firebase database, which brings us to the next step.

D. Setup the database for the android application

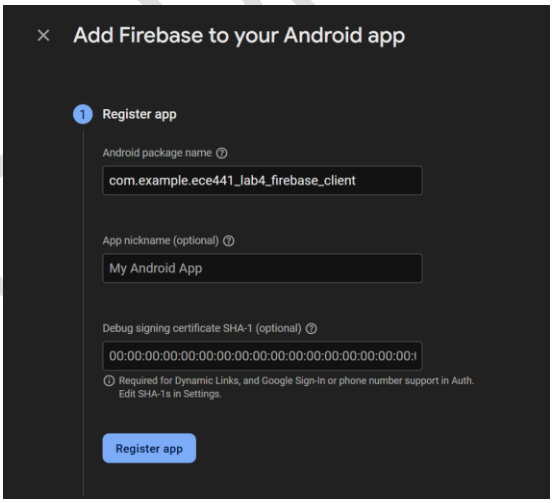
Adding Firebase to your app involves tasks both in the Firebase console and in your open Android project (for example, you download Firebase config files from the console, then move them into your Android project).

To use Firebase in your Android app, you need to register your app with your Firebase project. Registering your app is often called "adding" your app to your project.

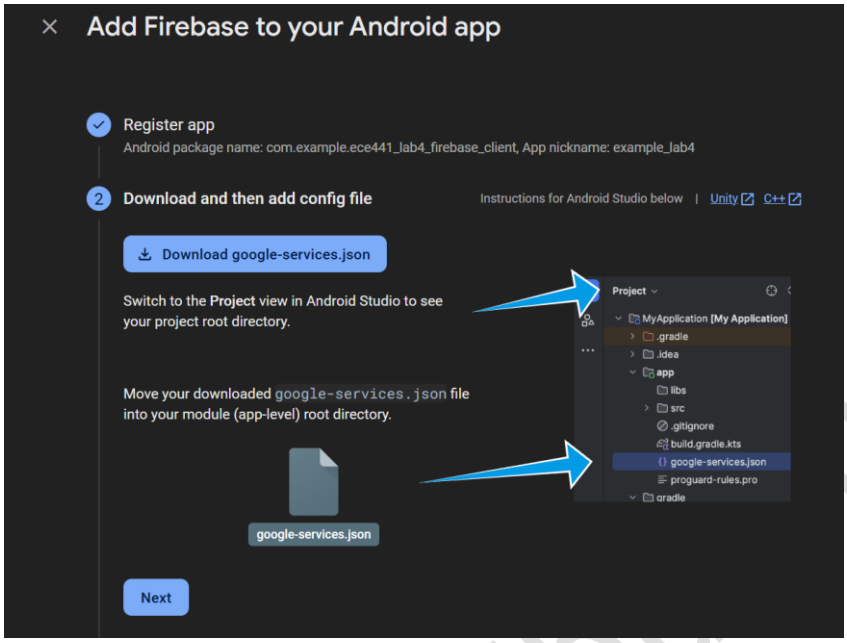
1. Go to the Firebase console.
2. In the center of the project overview page, click the Android icon or Add app to launch the setup workflow.
3. Enter your app's **applicationId** in the Android package name field. You can find and edit the **applicationId** in your android project.



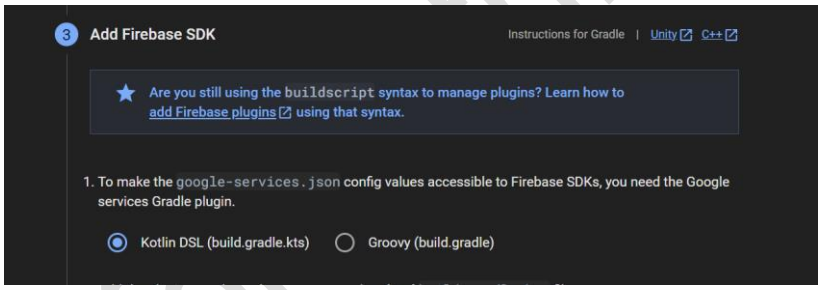
4. Click Register app.



5. Add the Firebase Android configuration file to your app:



6. Add firebase SDK to the android project. (The dependencies are already added in the sample project)



7. Wait for Gradle to synchronize all the dependencies.

E. (Option 1) Running the Android application on a cellphone

Set up your device as follows:

1. Connect your device to your development machine with a USB cable. If you developed on Windows, you might need to install the appropriate USB driver for your device.
2. Perform the following steps to enable **USB debugging** in the **Developer options** window:
    a. Open the **Settings** app.
    b. If your device uses Android v8.0 or higher, select **System**. Otherwise, proceed to the next step.
    c. Scroll to the bottom and select **About phone**.
    d. Scroll to the bottom and tap **Build number** seven times.

e. Return to the previous screen, scroll to the bottom, and tap **Developer options**.

f. In the **Developer options** window, scroll down to find and enable **USB debugging**.

Run the app on your device as follows:

1. In Android Studio, select your app from the run/debug configurations drop-down menu in the toolbar.
2. In the toolbar, select the device that you want to run your app on from the target device drop-down menu.

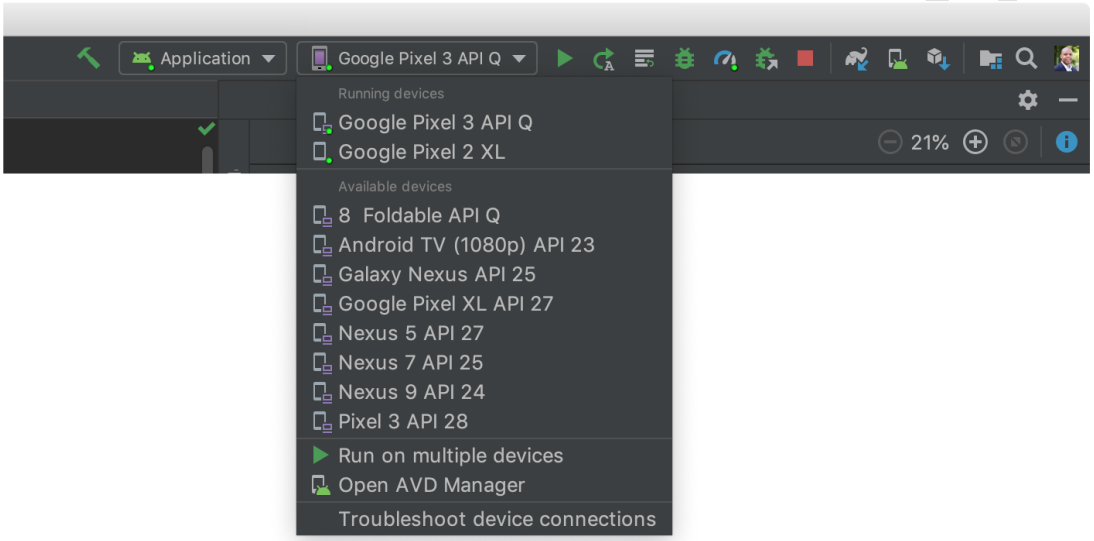


Figure 4 Target Device Dropdown menu

3. Click **Run**  . Android Studio installs your app on your connected device and starts it.

F. (Option 2) Running the android application in an emulator

Run the app on an emulator as follows:

1. In Android Studio, create an Android Virtual Device (AVD) that the emulator can use to install and run your app.
2. In the toolbar, select your app from the run/debug configurations drop-down menu.
3. From the target device drop-down menu, select the AVD that you want to run your app on.

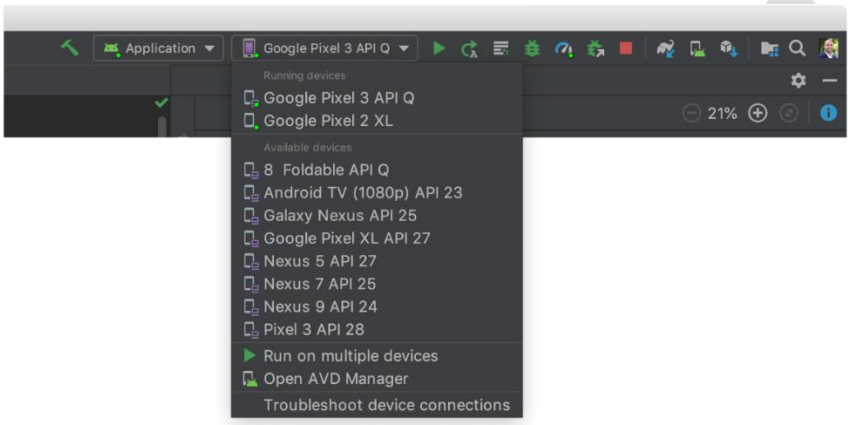


Figure 6 Target device drop-down menu

4. Click **Run**  .

   Android Studio installs the app on the AVD and starts the emulator.



Figure 7 Figure 4 Running the android application on an Android emulator

Not for Re

3. Discussion

In your lab report, you are required to answer **all the questions** in this section.

1. Full comment on the scripts used in this experiment (python and android).
2. On default, the database is not public. How did you make the database accessible for the Raspberry Pi and the Android application? Is the database vulnerable the way you do it?
3. Discuss how cloud databases could benefit or extend your own project.
4. On top of the firebase that we used in this experiment, find at least two other databases and discuss their differences.
5. An embedded system can have different topologies according to its purpose, service model, security requirement and reliability needs. Looking at a different arrangement of the system we designed in this experiment, as shown in Figure 8, please discuss about this design's rationality. Hint: in this arrangement the Raspberry Pi is only connected to the cloud when the Cellphone is physically around the Raspberry Pi, and a Raspberry Pi can be used in a portable device.

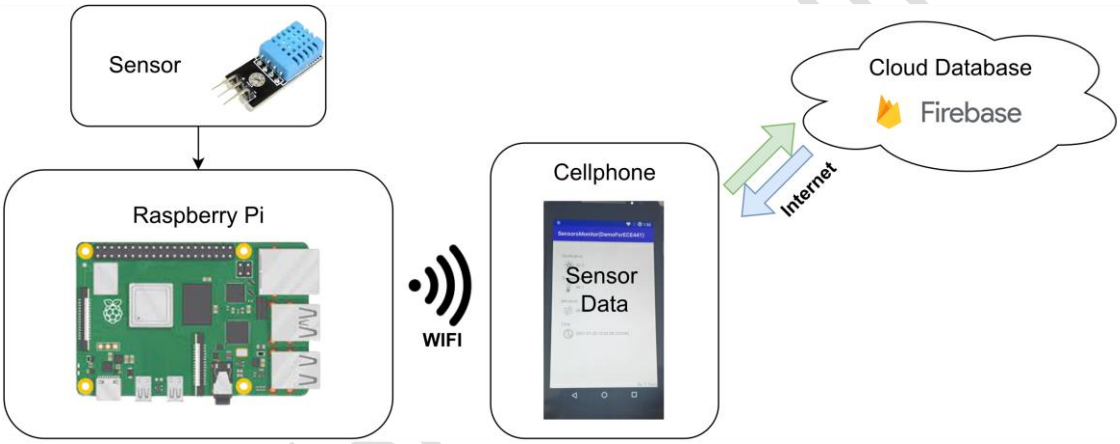


Figure 8 An alternative arrangement of the system we designed

4. References

[1] Raspberry Pi hardware documents
https://www.raspberrypi.org/documentation/hardware/raspberrypi/README.md
[2] Common Linux commands
https://www.raspberrypi.org/documentation/linux/usage/commands.md
[3] Python 3 tutorial
https://docs.python.org/3/tutorial/
[4] DHT22 Library
https://github.com/RobTillaart/Arduino/tree/master/libraries/DHTlib
[5] DHT22 Datasheet
https://cdn-learn.adafruit.com/downloads/pdf/dht.pdf
[6] Firebase Introduction
https://firebase.google.com/docs/database

Not for distribution

[7] Firebase Beginner's Guide

https://www.youtube.com/watch?v=9kRgVxULbag

[8] Android SDK Introduction

https://developer.android.com/guide

[9] Quick Android SDK Guide

https://www.youtube.com/watch?v=kMI2jy-WlGM

[10] 15 Hour Android SDK Course

https://www.youtube.com/watch?v=fis26HvvDII

https://www.youtube.com/watch?v=RcSHAkpwXAQ

5. Appendix

A. Raspberry Pi code

```
import pyrebase
from datetime import datetime
import time

import board
import adafruit_ahtx0

# Firebase configuration. Create a firebase web app in Firebase Dashboard and link it here.
config = {
    "apiKey": "",
    "authDomain": "",
    "projectId": "",
    "storageBucket": "",
    "messagingSenderId": "",
    "appId": "",
    "measurementId": "", #ignore this
    "databaseURL": ""
}

firebase = pyrebase.initialize_app(config)
database = firebase.database()

# Initialize the AHT20 sensor
sensor = adafruit_ahtx0.AHTx0(board.I2C())

while True:
    now = datetime.now()

    try:
        # Read temperature and humidity from the AHT20 sensor
        temp = sensor.temperature
        hum = sensor.relative_humidity
    except Exception as error:
        print(error)
        print("bad measurement, retrying")
        continue

```

Not for Distribution

```
    data = {
        "timestamp": str(now),
        "humidity": str(hum),
        "temperature": str(temp)
    }

    # Uncomment to send data to Firebase
    #database.child("SensorDatabase").update(data)

    print(data.get("timestamp") + "\tMoisture: " + data.get("humidity") + "\t\tTemperature: " +
data.get("temperature"))
    time.sleep(2) #update every two seconds

```

B. Android code

```
package com.example.myapplication;

import android.os.Bundle;
import android.util.Log;
import android.widget.TextView;
import androidx.activity.EdgeToEdge;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import com.google.firebase.FirebaseApp;
import com.google.firebase.database.DataSnapshot;
import com.google.firebase.database.DatabaseError;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.database.ValueEventListener;
import java.util.logging.Logger;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        FirebaseApp.initializeApp(this);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findView-
ById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCom-
pat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, system-
Bars.right, systemBars.bottom);
            return insets;
        });
        Thread thread = new Thread(){
            @Override
            public void run() {
                try {
                    basicReadWrite();
```

```
                } catch (Exception e) {
                    e.printStackTrace();
                }

                return;
            };
        };
        thread.start();

    }
    public void basicReadWrite() {
        FirebaseDatabase database = FirebaseDatabase.getInstance();
        DatabaseReference myFirebase = database.getReference("Sensor-
Database");

        TextView timeUI=findViewById(R.id.text_time);
        TextView tempUI=findViewById(R.id.text_temp);
        TextView humidUI=findViewById(R.id.text_humid);

        myFirebase.addValueEventListener(new ValueEventListener() {
            @Override
            public void onDataChange(@NonNull DataSnapshot dataSnapshot)
{
                String time_stamp = dataSnapshot.child("timestamp").get-
Value(String.class);
                String humidity = dataSnapshot.child("humidity").get-
Value(String.class);
                String temperature = dataSnapshot.child("tempera-
ture").getValue(String.class);
                Log.d("MainActivity", time_stamp + " Humidity: " + hu-
midity + " Temperature: " + temperature);
                runOnUiThread(() -> {
                    // Stuff that updates the UI
                    timeUI.setText(time_stamp);
                    if (temperature == null) throw new AssertionError();
                    tempUI.setText(temperature.concat("°C"));
                    if (humidity == null) throw new AssertionError();
                    humidUI.setText(humidity.concat("%"));
                });

            }

            @Override
            public void onCancelled(@NonNull DatabaseError error) {
                Log.w("MainActivity", "Failed to read value.", er-
ror.toException());
            }
        });
    }
}
```